

0.18. Avancer avec `deplace_x`

`deplace_x(colonne, ligne, tuile, pas)` fait tout le 0.17 en une ligne : la lettre avance de 5 cases.

1. Ce qu'il faut comprendre

C'est le programme du 0.17, mais tout le travail (le chronomètre, l'effacement, le pas de plus, la lettre reposée) est fait par **une seule fonction de la console** : `deplace_x`.

Ce qui est nouveau ici : `deplace_x(colonne, ligne, tuile, pas)`. Elle pose la tuile en (`colonne`, `ligne`), puis la fait avancer d'une case vers la droite tous les quarts de seconde (15 images), **pas** fois.

Pourquoi `x = deplace_x(...)` ? Quand on écrit `deplace_x(x, ...)`, la fonction ne reçoit pas la variable `x` elle-même, mais une **copie** de sa valeur (0). Elle fait bouger la lettre en changeant sa copie, jamais notre `x`. À la fin, elle **rend** (`return`) la colonne d'arrivée, 5, et `x = ...` range ce 5 dans `x`.

Sans le `x =`, en C ordinaire, la lettre bougerait bien à l'écran, mais `x` vaudrait encore 0 : le programme croirait la lettre toujours au départ. Le cours suivant montre quand cela pose problème.

Elle bloque, comme `attendre()` : pendant le voyage, rien d'autre ne tourne. Au bord de l'écran (colonne 19), elle s'arrête au lieu de sortir.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction `deplace\_x\(\)` — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1070 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage – la mise en route de la console
deplace_x()    158 o ← main() – une fonction fournie par le langage
main()         27 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases – attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() – efface des cases
CopierTuiles   18 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles – la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 2    26 o sur place, dans deplace_x() + AdresseCase (10 o)
```

```
effacer()   x 1   15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()     x 2    6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)
Tes fonctions appelées :
deplace_x() x 1   18 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5; // le nombre de cases à parcourir
```

3. UNE VARIABLE — X

```
uint8_t x = 0; // la colonne de la lettre : elle part de 0 (tout à gauche)
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La ligne, morceau par morceau :
    //
    // x = deplace_x(x, 0, ALPHABET[0], pas);
    // | | | | |
    // | | | | | +-- pas : combien de cases (5)
    // | | | | | +----- tuile : ce qui bouge, la lettre A
    // | | | | | +----- ligne : 0, tout en haut
    // | | | | | +----- colonne : d'où la lettre part.
    // | | | | | La fonction reçoit une COPIE
    // | | | | | de x (sa valeur, 0), pas x.
    // | +--- deplace_x fait avancer la lettre, puis REND (return) la
    // | colonne d'arrivée : 5.
    // +----- « x = » range ce 5 dans x.
    //
    // POURQUOI « x = » ? Une fonction n'a qu'une COPIE de x : elle ne peut pas
    // le changer. Pour garder le résultat, on le RANGE nous-mêmes : x = ...
    x = deplace_x(x, 0, ALPHABET[0], pas); // après cette ligne : x vaut 5

    while (true) {
        image(); // le programme continue ; la lettre, elle, reste en colonne 5
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui avance de 5 cases vers la droite, puis s'arrête en colonne 5.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ x a reçu la colonne d'arrivée ($x = 5$)
- ✓ le A est en colonne 5, seul sur sa ligne (« A »)

Ensuite : **0.18.1. Avancer avec `deplace_x` — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++